What2Cook

Documentación de arquitectura del sistema y modelo de objetos

Grupo 2 — Entrega 2

Grupo 2

Valentín Joel Romero Monteagudo
Gustavo Andrés Contardi
Matt Zander

1. Visión general

What2Cook es una aplicación web de tres capas (presentación, lógica de negocio, datos) construida sin frameworks, lo que implica que cada capa se implementa manualmente con tecnologías base.

Tecnologías por capa

- Frontend: HTML5, CSS3, JavaScript (Vanilla ES6+)
- Backend: PHP 8.x puro
- Base de datos: PostgreSQL
- API externa: Spoonacular (consumida desde el backend vía cURL)
- Servidor web: Apache
- Autenticación: Sesiones PHP nativas
- Desarrollo: Docker y Docker Compose para desacoplar las dependencias del proyectos de los sistemas de desarrollo.

---

2. Estructura de directorios

Sseparación entre public/ y api/: Apache debe estar configurado para que el directorio raíz del virtual host apunte a public/, de modo que los archivos PHP del backend no sean accesibles directamente desde el browser.

Directorio / Archivo	Descripción
what2cook/	Raíz del proyecto
public/	Archivos servidos directamente al cliente (HTML, CSS, JS, imágenes)
public/index.php	Punto de entrada principal. Renderiza la estructura base del sitio
public/pages/	Una subcarpeta por sección: catalogo/, asistente-cocina/, asistente-dietas/, dietas/, acerca/
public/assets/	CSS (styles.css), JS por página, imágenes estáticas
api/	Backend PHP. Contiene todos los endpoints REST
api/index.php	Router central: recibe todas las peticiones y las despacha al controlador correcto
api/controllers/	Un archivo por recurso: RecipeController.php, UserController.php, PlanController.php, etc.
api/services/	Lógica de negocio desacoplada de los controladores: SpoonacularService.php, MealPlanService.php, etc.

Directorio / Archivo	Descripción
api/models/	Clases que representan entidades del dominio y encapsulan las consultas SQL: User.php, Plan.php, Favorite.php
api/middleware/	AuthMiddleware.php: verifica que exista una sesión activa antes de llegar al controlador
api/config/	database.php (conexión PDO), env.php (carga variables de entorno), cors.php
api/helpers/	Funciones utilitarias reutilizables: response.php (estandariza JSON)
.env	Variables sensibles: DB_HOST, DB_PASS, SPOONACULAR_KEY. Archivo no versionado.
.htaccess	Rewrite rules de Apache: redirige todo hacia api/index.php y public/index.php según la ruta

Nota sobre .htaccess: es el único mecanismo de routing disponible sin framework. Cada sección del sitio necesita su propia regla de rewrite para que las URLs amigables (por ejemplo, /catalogo/42) funcionen correctamente en Apache.

---

3. Arquitectura del backend (PHP)

El backend sigue el patrón MVC simplificado (sin ORM ni sistema de templates). Todas las peticiones HTTP a /api/* pasan por un único punto de entrada.

3.1 Flujo de una petición

- El cliente (browser) realiza una petición HTTP a /api/recipes?diet=vegan.
- Apache aplica las rewrite rules del .htaccess y redirige hacia api/index.php.
- api/index.php parsea la URL y el método HTTP, e invoca al controlador correspondiente.
- Si la ruta requiere autenticación, se ejecuta AuthMiddleware.php antes de llegar al controlador.
- El controlador llama al servicio o modelo que corresponda.
- Si hace falta datos externos, el servicio llama a Spoonacular vía cURL.
- El controlador construye la respuesta usando response.php y devuelve JSON.

3.2 Separación de responsabilidades

- Controladores: solo reciben la petición, invocan servicios/modelos y devuelven respuesta. Sin lógica de negocio.
- Servicios: implementan la lógica de negocio compleja (por ejemplo, optimizar ingredientes para Meal Prep, recalcular los criterios de búsqueda al modificar los objetivos nutricionales dentro de un plan). Son independientes del protocolo HTTP.
- Modelos: encapsulan el acceso a la base de datos usando PDO con prepared statements. Nunca construir SQL con concatenación de strings directa.

- SpoonacularService.php: único punto de contacto con la API externa. Centralizar aquí facilita el manejo de errores, el cacheo futuro y el reemplazo de la API si fuera necesario.

3.3 Autenticación

El sistema usa sesiones PHP nativas. El servidor mantiene el estado de cada usuario autenticado en memoria o en archivos del sistema.

- Al loguearse, el backend ejecuta session_start() y almacena el ID y email del usuario en $_SESSION. PHP genera automáticamente una cookie de sesión (PHPSESSID) que el browser envía en cada petición siguiente.
- AuthMiddleware.php ejecuta session_start() y verifica que $_SESSION['user_id'] exista antes de ejecutar cualquier controlador protegido. Si no existe, devuelve HTTP 401.
- Al cerrar sesión, el backend llama a session_destroy() que invalida la sesión en el servidor inmediatamente.

Consideración importante con Docker: los archivos de sesión se almacenan en /var/lib/php/sessions dentro del contenedor y se pierden al reiniciarlo. Es obligatorio declarar un volumen persistente para ese directorio en el docker-compose.yml.

3.4 Comunicación con Spoonacular

Spoonacular se consume exclusivamente desde el backend, nunca directamente desde JavaScript del frontend. Esto protege la API key y permite aplicar rate limiting y cacheo del lado del servidor.

- SpoonacularService.php construye la URL del endpoint correspondiente, agrega la apiKey desde .env, y realiza la petición con cURL.
- La respuesta se parsea con json_decode() y se transforma a un DTO (Data Transfer Object) antes de devolvérsela al controlador.
- Spoonacular tiene límites en su plan gratuito (100 puntos de procesamiento/día). Se recomienda cachear respuestas frecuentes en la base de datos local.

---

4. Arquitectura del frontend

El frontend es HTML/CSS/JS. No hay ningún framework de componentes, ni sistema de build.

4.1 Comunicación con el backend

Todas las llamadas al backend se realizan con la Fetch API en JavaScript asíncrono.

- Las funciones de fetch se organizan en módulos JS por recurso: api/recipes.js, api/auth.js, api/plans.js.
- El browser envía automáticamente la cookie de sesión (PHPSESSID) en cada petición al mismo dominio. No es necesario adjuntar ningún header manualmente en el frontend.
- Los errores HTTP (401, 403, 404, 500) deben manejarse explícitamente: mostrar mensajes de error al usuario, y redirigir al login ante un 401.

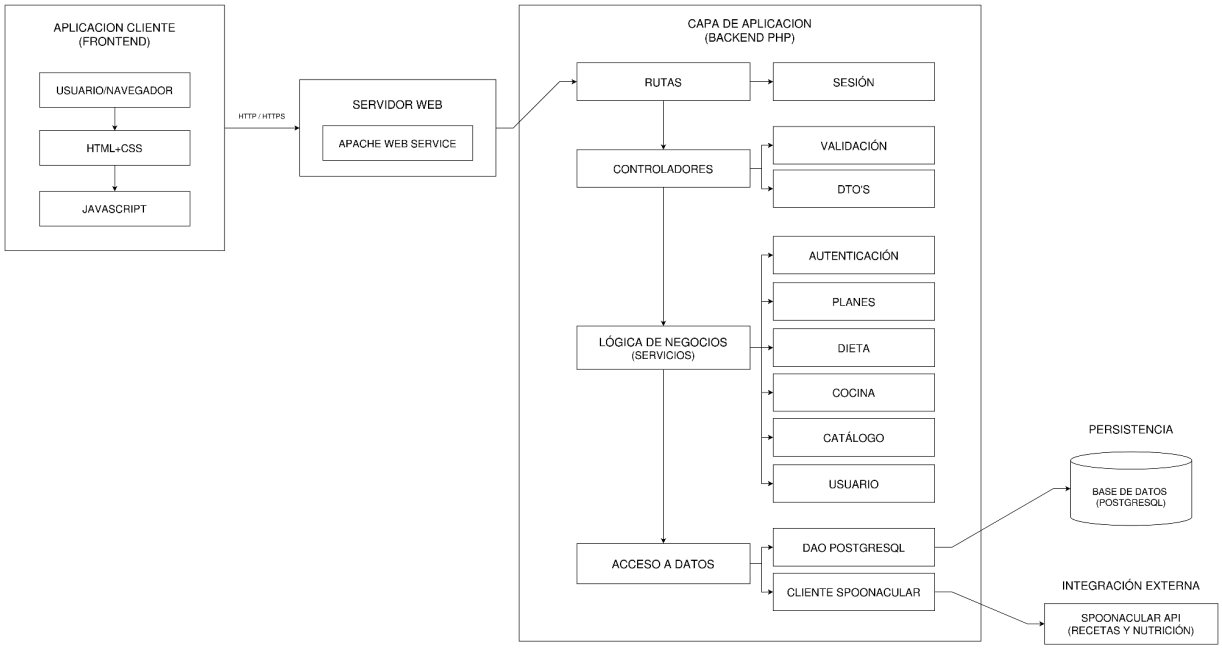
4.2 Estado del cliente

Sin framework, el "estado" de la aplicación (usuario logueado, plan activo, etc.) se gestiona manualmente:

- El estado de autenticación vive en el servidor (en la sesión PHP). El frontend puede hacer una petición a un endpoint como /api/auth/me al cargar para saber si hay una sesión activa y obtener los datos básicos del usuario.
- Al cargar cada página se consulta ese endpoint para mostrar u ocultar elementos de la zona privada según si la respuesta es 200 o 401.
- Al cerrar sesión, el frontend llama a /api/auth/logout y redirige al home. No hay nada que limpiar en el browser.

---

5. Diagrama de arquitectura



---

5. Modelo de objetos

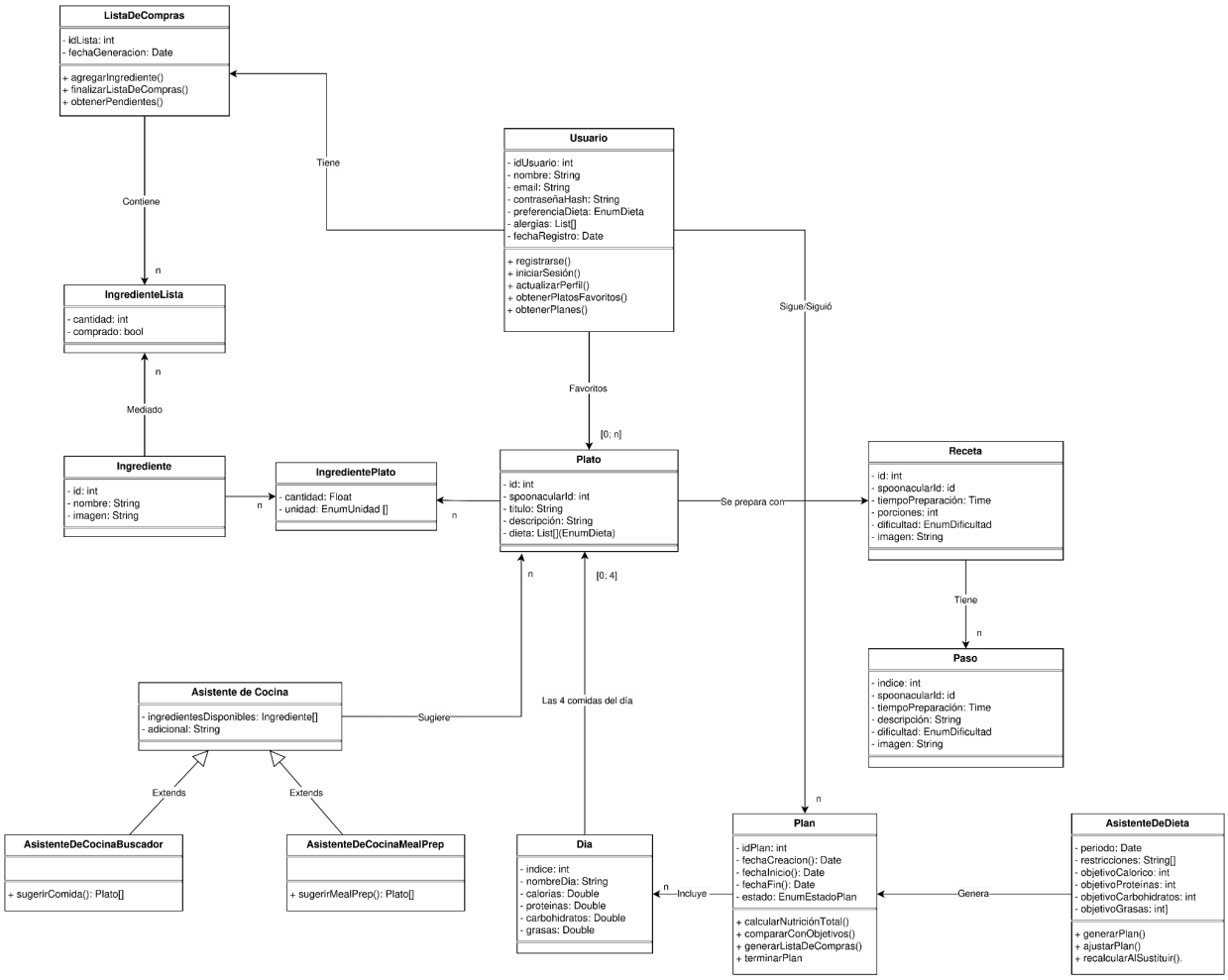
El sistema se organiza alrededor del **Usuario**, que es el punto de entrada a toda la funcionalidad privada. Un usuario tiene platos favoritos, planes de comida y listas de compras.

El **Plato** es la entidad central del dominio. Representa una comida completa con su información nutricional y las dietas que cumple. Cada plato se prepara siguiendo una **Receta**, que contiene los **Pasos** ordenados con sus instrucciones. La separación entre Plato y Receta permite que el sistema maneje por un lado el resultado final (qué se come, con qué macros) y por otro el proceso de preparación (cómo se cocina).

Cada plato está compuesto por ingredientes a través de **IngredientePlato**, que registra la cantidad y unidad específica de cada **Ingrediente** en ese plato. Esta relación intermedia es la que permite calcular las cantidades para la lista de compras.

El sistema ofrece dos asistentes. El **Asistente de Cocina** recibe los ingredientes disponibles del usuario y sugiere platos. Se especializa en dos modos mediante herencia: el **Buscador** sugiere una comida única y el **MealPrep**, dado uno o varios alimentos en cantidad, sugiere un conjunto de comidas para la semana. El **Asistente de Dieta** parte de objetivos nutricionales (calorías, macros, restricciones) y genera un **Plan** completo.

El **Plan** organiza las comidas en el tiempo. Contiene múltiples **Días**, y cada día tiene hasta 4 comidas (desayuno, almuerzo, merienda, cena), cada una asignada a un plato. El día calcula automáticamente los totales nutricionales sumando los macros de sus platos. El plan puede comparar esos totales contra los objetivos del usuario y generar una **Lista de Compras**.

La **Lista de Compras** consolida todos los ingredientes necesarios para un plan a través de **IngredienteLista**, que registra la cantidad total de cada ingrediente y su estado (comprado o pendiente).

4.2 Diagrama de clases (primera versión)